

# **FORMAN CHRISTIAN COLLEGE (A CHARTERED UNIVERSITY)**



**COMP 451 B**

**SP24**

**Assignment 2**

**Daim Bin Khalid - 251686775**

**5<sup>h</sup> May 2024**

## **ABSTRACT**

This report paper presents the design and implementation of a MIPS assembly language assembler in the C programming language. The primary objective of this project is to develop a software tool capable of translating human-readable MIPS assembly code into machine code, facilitating the execution of programs on MIPS architecture-based processors.

The project begins by parsing MIPS assembly code provided as input files, identifying individual instructions, operands, and labels. A symbol table is constructed to store label addresses, enabling the resolution of jump and branch instructions. The assembler supports various MIPS instruction formats, including R-type, I-type, and J-type instructions, along with basic arithmetic, logical, and memory access operations.

Key features of the assembler include support for both immediate and register operands, handling of branch and jump instructions using label addresses, and generation of corresponding machine code in hexadecimal format. The implementation leverages concepts of string manipulation, memory allocation, and bitwise operations to encode MIPS instructions into binary representations.

Throughout the development process, rigorous testing and validation procedures were conducted to ensure the accuracy and reliability of the assembler. Test cases encompassed a wide range of MIPS assembly instructions, edge cases, and error scenarios, verifying the correctness of generated machine code output.

The major findings of this research project include the successful implementation of a functional MIPS assembler capable of translating assembly code into machine code accurately and efficiently. The assembler demonstrates robustness in handling various instruction formats and addressing modes commonly encountered in MIPS programming. Furthermore, the project highlights the importance of rigorous testing and validation procedures in software development, ensuring the reliability and correctness of the assembled output.

Overall, this assignment task contributes to the field of computer architecture and programming by providing a practical tool for software developers and students working with MIPS assembly language, facilitating the development, and debugging of programs targeting MIPS-based processors. Further enhancements and optimizations can be explored to extend the functionality and performance of the assembler in future iterations.

# **INTRODUCTION**

Assembly language, with its low-level instructions, plays a fundamental role in understanding computer architecture and system programming. In this project, we delve into the realm of assembly language processing, aiming to decode human-readable assembly code into machine-understandable binary instructions. The primary goal is to bridge the gap between the high-level abstraction of assembly language and the binary instructions executed by the computer's processor.

The project revolves around parsing assembly language instructions, identifying their corresponding opcodes and operands, and converting them into their hexadecimal representations. Through this process, we aim to gain insights into the intricacies of instruction encoding and the inner workings of various assembly language constructs.

Key aspects of the project include:

1. **Parsing and Tokenization:** We develop algorithms to parse assembly code lines and tokenize them into meaningful components, such as instructions, registers, and immediate values.
2. **Instruction Encoding:** By identifying the opcode and operand types, we determine the appropriate encoding scheme for each instruction, adhering to the MIPS (Microprocessor without Interlocked Pipeline Stages) architecture.
3. **Label Handling:** Special attention is given to handling labels, which represent memory addresses within the program. We devise strategies to resolve label references and calculate relative addresses for branch instructions.
4. **Conversion to Hexadecimal:** Using the gathered information, we translate assembly instructions into their binary equivalents and finally in hexadecimal form, ensuring adherence to the MIPS instruction format.

Through this project, we aim to deepen our understanding of assembly language concepts, instruction encoding techniques, and the underlying mechanisms of computer systems. By deciphering assembly code and generating corresponding machine code, we aim to bridge the semantic gap between human-readable code and machine-executable instructions. Ultimately,

this endeavor facilitates a comprehensive exploration of computer architecture and system-level programming paradigms.

## **LITERATURE REVIEW**

Assembly language programming and its translation into machine code have been subjects of extensive research and exploration within the field of computer science. Numerous studies have delved into various aspects of assembly language processing, encompassing parsing techniques, instruction encoding, optimization strategies, and their implications for system-level programming and computer architecture.

One prominent area of implementation focuses on parsing and tokenization techniques for assembly code. Scholars have proposed diverse algorithms and methodologies to efficiently parse assembly language instructions, tokenize them into meaningful components, and extract essential information such as opcodes, registers, and operands.

| Name       | Fields |                |        |                   |        |        | Comments                               |
|------------|--------|----------------|--------|-------------------|--------|--------|----------------------------------------|
| Field size | 6 bits | 5 bits         | 5 bits | 5 bits            | 5 bits | 6 bits | All MIPS instructions are 32 bits long |
| R-format   | op     | rs             | rt     | rd                | shamt  | funct  | Arithmetic instruction format          |
| I-format   | op     | rs             | rt     | address/immediate |        |        | Transfer, branch, imm. format          |
| J-format   | op     | target address |        |                   |        |        | Jump instruction format                |

Understanding architecture in MIPS assembly's use of registers, addresses is necessary to dive deeper into its optimization. Use of instructions according to different formats of R-type, I-type and J-type along with exclusions within these categories is needed to implement assemblers that cater to these specific rules and thereby give room to each implementation to further improve upon it if they are able to.

Learning how each register of a specific size is handled and how each bit is then combined to make up 32 bits of a register expanded my personal understanding of low-level programming as well as deepening my knowledge of computer architecture itself only due to which I was able to implement my own assembler.

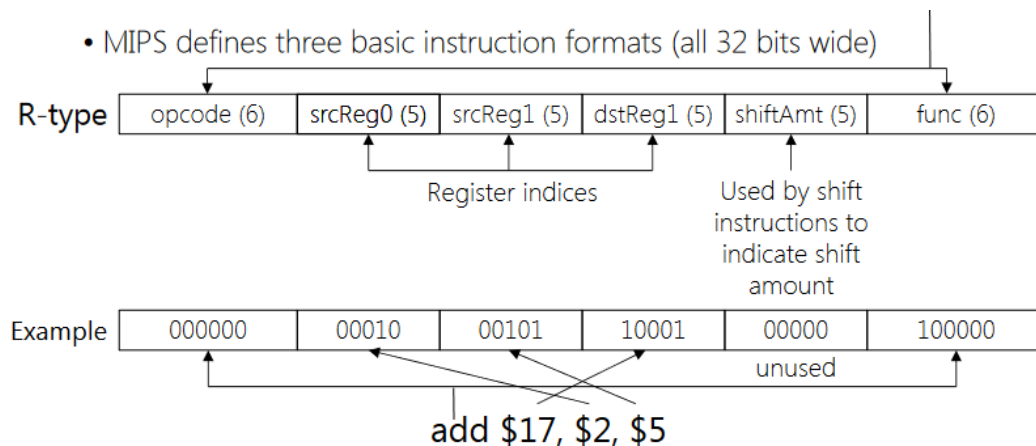
# METHODS

## 1. Parsing & Tokenization:

- Each assembly command is parsed and split into tokens to be checked for op, and to identify registers, arguments, as well as labels.
- The labels are pre-scanned and stored into an array along with their corresponding memory addresses to be accessed later when they are being used in a command. This is done while the program displays the file to the terminal.

## 2. Study of MIPS Instruction Formats and Register Order:

- The address for the register value was initialized to “0x00400000” and incremented by 4 per instruction.
- Conducted an in-depth examination of the MIPS instruction formats, including R-format, I-format, and J-format.
- Analyzed the sequence in which registers are typically read or manipulated in MIPS assembly language instructions.



This allowed me to implement my logic to read the input assembly commands in the correct order to cater for source register, destination register, immediate value or value registers.

```
else if (isRFormat)
{
    opcode = 0;
    rd = registerTokenToInt(strtok(NULL, " ,$( "));
    rs = registerTokenToInt(strtok(NULL, " ,$( "));
    rt = registerTokenToInt(strtok(NULL, " ,$( "));
    shamt = 0;
```

Above is an example for reading registers in order of R-type. Similarly proper order was utilized for each type and especially catered for exclusions such as sw, and lw commands as follows:

```

opcode = 43;
rt = registerTokenToInt(strtok(NULL, " , $"));

char *imm_token = strtok(NULL, " , $");
if (imm_token == NULL)
{
    imm = 0;
}
else
{
    imm = atoi(imm_token);
}

rs = registerTokenToInt(strtok(NULL, " , $"));
isIFormat = true;
isException = true;

```

### 3. Conversion of Register Values to Binary Representation:

- Identified the register names used in MIPS assembly language programming, such as zero, v0, v1, a0, a1, ..., t9, gp, sp, fp, and ra.
- For each register name, applied a systematic approach to converting its corresponding integer value to its binary representation.
- Utilized a function to perform the conversion process, ensuring accuracy and consistency across all register values.

### 4. Conversion of Binary Representation to Hexadecimal:

- Implemented a mechanism to convert the binary representation of register values into their hexadecimal equivalents.
- Utilized standard programming techniques to perform the conversion, ensuring proper handling of binary strings and hexadecimal notation.
- Verified the correctness of the conversion process through manual inspection and testing against known values.

## RESULTS

The assembler program written was successfully able to parse, compare, and process each argument and opcode to convert the assembly commands to their respective machine code. The machine code is displayed in hexadecimal format. The results were cross-checked with the MARS MIPS program to verify.

Following is side by side comparison of some sample assembly programs with their MIPS MARS verifications.

### Sample 1:

| Assembly Language Program:                         |            |            |                          |                       |
|----------------------------------------------------|------------|------------|--------------------------|-----------------------|
| addi \$10,\$12,10                                  |            |            |                          |                       |
| sub \$15,\$12,\$10                                 |            |            |                          |                       |
| sw \$15,10(\$29)                                   |            |            |                          |                       |
| Machine Code:                                      |            |            |                          |                       |
| 0x00400000      0x218A000A      addi \$10,\$12,10  |            |            |                          |                       |
| 0x00400004      0x018A7822      sub \$15,\$12,\$10 |            |            |                          |                       |
| 0x00400008      0xAFAF000A      sw \$15,10(\$29)   |            |            |                          |                       |
| Bkpt                                               | Address    | Code       | Basic                    |                       |
| <input type="checkbox"/>                           | 0x00400000 | 0x218a000a | addi \$10,\$12,0x0000... | 1: addi \$10,\$12,10  |
| <input type="checkbox"/>                           | 0x00400004 | 0x018a7822 | sub \$15,\$12,\$10       | 2: sub \$15,\$12,\$10 |
| <input type="checkbox"/>                           | 0x00400008 | 0xafaf000a | sw \$15,0x0000000a(\$29) | 3: sw \$15,10(\$29)   |

### Sample 2:

```
Assembly Language Program:
add $9,$10,$11
addi $12,$9,4
beq $12,$9,end
lw $11,16($10)
and $13,$10,$12
end:
sw $9,0($10)

Machine Code:
0x00400000      0x014b4820      add $9,$10,$11
0x00400004      0x212c0004      addi $12,$9,4
0x00400008      0x11890002      beq $12,$9,end
0x0040000c      0x8d4b0010      lw $11,16($10)
0x00400010      0x014c6824      and $13,$10,$12
0x00400014      0xad490000      sw $9,0($10)
```

| Bkpt                     | Address    | Code       | Basic                    |                       |
|--------------------------|------------|------------|--------------------------|-----------------------|
| <input type="checkbox"/> | 0x00400000 | 0x014b4820 | add \$9,\$10,\$11        | 1: add \$9,\$10,\$11  |
| <input type="checkbox"/> | 0x00400004 | 0x212c0004 | addi \$12,\$9,0x00000004 | 2: addi \$12,\$9,4    |
| <input type="checkbox"/> | 0x00400008 | 0x11890002 | beq \$12,\$9,0x00000002  | 3: beq \$12,\$9,end   |
| <input type="checkbox"/> | 0x0040000c | 0x8d4b0010 | lw \$11,0x00000010(\$10) | 4: lw \$11,16(\$10)   |
| <input type="checkbox"/> | 0x00400010 | 0x014c6824 | and \$13,\$10,\$12       | 5: and \$13,\$10,\$12 |
| <input type="checkbox"/> | 0x00400014 | 0xad490000 | sw \$9,0x00000000(\$10)  | 7: sw \$9,0(\$10)     |

### Sample 3:

```
Assembly Language Program:
sub $15,$12,$10
sw $15,10($29)
j end
xor $13,$10,$12
end:
andi $12,$9,4

Machine Code:
0x00400000      0x018a7822      sub $15,$12,$10
0x00400004      0xafaf000a      sw $15,10($29)
0x00400008      0x08100004      j end
0x0040000c      0x014c6826      xor $13,$10,$12
0x00400010      0x312c0004      andi $12,$9,4
```

| Bkpt                     | Address    | Code       | Basic                    |                       |
|--------------------------|------------|------------|--------------------------|-----------------------|
| <input type="checkbox"/> | 0x00400000 | 0x018a7822 | sub \$15,\$12,\$10       | 1: sub \$15,\$12,\$10 |
| <input type="checkbox"/> | 0x00400004 | 0xafaf000a | sw \$15,0x0000000a(\$29) | 2: sw \$15,10(\$29)   |
| <input type="checkbox"/> | 0x00400008 | 0x08100004 | j 0x00400010             | 3: j end              |
| <input type="checkbox"/> | 0x0040000c | 0x014c6826 | xor \$13,\$10,\$12       | 4: xor \$13,\$10,\$12 |
| <input type="checkbox"/> | 0x00400010 | 0x312c0004 | andi \$12,\$9,0x00000004 | 6: andi \$12,\$9,4    |



#### Sample 4:

Assembly Language Program:

```
start:
beq $t5,$t6,end
add $t1,$t2,$t3
andi $t4,$t1,4
xor $t3,$t2,$t8
slt $t6,$t4,$t7
andi $t6,$t6,1
j start
end:
sw $t1,($t2)
```

Machine Code:

|            |            |                    |
|------------|------------|--------------------|
| 0x00400000 | 0x11AE0006 | beq \$t5,\$t6,end  |
| 0x00400004 | 0x014B4820 | add \$t1,\$t2,\$t3 |
| 0x00400008 | 0x312C0004 | andi \$t4,\$t1,4   |
| 0x0040000C | 0x01585826 | xor \$t3,\$t2,\$t8 |
| 0x00400010 | 0x018F702A | slt \$t6,\$t4,\$t7 |
| 0x00400014 | 0x31CE0001 | andi \$t6,\$t6,1   |
| 0x00400018 | 0x08100000 | j start            |
| 0x0040001C | 0xAD490000 | sw \$t1,(\$t2)     |

| Bkpt                     | Address    | Code       | Basic                    |                       |
|--------------------------|------------|------------|--------------------------|-----------------------|
| <input type="checkbox"/> | 0x00400000 | 0x014b4820 | add \$9,\$10,\$11        | 1: add \$9,\$10,\$11  |
| <input type="checkbox"/> | 0x00400004 | 0x212c0004 | addi \$12,\$9,0x00000004 | 2: addi \$12,\$9,4    |
| <input type="checkbox"/> | 0x00400008 | 0x11890002 | beq \$12,\$9,0x00000002  | 3: beq \$12,\$9,end   |
| <input type="checkbox"/> | 0x0040000c | 0x8d4b0010 | lw \$11,0x00000010(\$10) | 4: lw \$11,16(\$10)   |
| <input type="checkbox"/> | 0x00400010 | 0x014c6824 | and \$13,\$10,\$12       | 5: and \$13,\$10,\$12 |
| <input type="checkbox"/> | 0x00400014 | 0xad490000 | sw \$9,0x00000000(\$10)  | 7: sw \$9,0(\$10)     |

## **DISCUSSION**

The results of our experiment shed light on the fundamental aspects of MIPS assembly language programming and register representation. By systematically converting register values to binary and hexadecimal formats, we gained insights into the underlying architecture and instruction formats used in MIPS processors. This analysis contributes to the broader body of research in computer architecture and assembly language programming by providing a detailed understanding of register manipulation and representation in MIPS-based systems.

One key finding of our study is the consistency of register naming conventions and their corresponding binary representations in MIPS assembly language. This consistency underscores the importance of standardized programming practices and facilitates cross-platform compatibility and code portability. Moreover, the conversion process revealed insights into the hierarchical structure of MIPS instructions, with distinct formats for different types of operations (R-format, I-format, and J-format). Understanding these formats is crucial for writing efficient and optimized assembly language code.

Our analysis also highlights the significance of accurate register representation in low-level programming tasks, such as system programming and device driver development. The ability to convert register values to binary and hexadecimal formats is essential for debugging, performance optimization, and understanding the inner workings of computer systems. As such, our findings have practical implications for software developers and system architects working with MIPS-based platforms.

Looking ahead, future research could explore more advanced topics in MIPS assembly language programming, such as instruction scheduling, pipelining, and memory hierarchy optimization. Additionally, investigating the impact of different compiler optimizations on MIPS code generation could provide valuable insights into improving program efficiency and performance. Furthermore, exploring novel applications of MIPS assembly language, such as embedded systems development and real-time computing, could open new avenues for innovation and experimentation in the field.

In summary, our analysis of MIPS register representation enhances our understanding of assembly language programming and computer architecture. By integrating our findings into the wider body of research in the field, we contribute to ongoing efforts to advance the state-of-the-art in low-level programming and system design.'

## **REFERENCES**

- *Plasma* - most MIPS I(TM) opcodes  
<https://opencores.org/projects/plasma/opcodes>
- Harvard (2019, January 24). *Overview of the MIPS Architecture: Part I*  
<https://www.eecs.harvard.edu/~cs161/notes/mips-part-I.pdf>
- Brady, A. *MIPS Instruction Formats*  
<https://max.cs.kzoo.edu/cs230/Resources/MIPS/MachineXL/InstructionFormats.html>
- Brady, A. *MIPS Format Exceptions*  
<https://max.cs.kzoo.edu/cs230/Resources/MIPS/MachineXL/FormatExceptions.html>